Programmeeropdracht Python OOP

Je wordt gevraagd een onderwijsbeheersysteem te ontwikkelen dat gedetailleerde informatie bijhoudt over personen, studenten, professoren, cursussen en inschrijvingen.

Deze informatie is cruciaal voor de dagelijkse werking van een onderwijsinstelling.

Systeemvereisten

Je systeem zal bestaan uit verschillende klassen, elk met specifieke attributen en methoden:

Address:

- Bevat attributen voor straat, stad, postcode en land.

Person:

- Houdt basisinformatie bij over een individu, inclusief voornaam, achternaam, geboortedatum, telefoonnummer en een lijst van Address objecten.
- Implementeer een method add_address voor het toevoegen van één of meerdere adressen (vb adres kot + thuisadres)
- Implementeer ook een method convert_string_to_date om de geboortedatum, die als string dient te worden ingevoerd, te testen op geldig formaat ('dd/mm/yyyy'), en:
  - indien het formaat geldig is, de datum te converteren naar een datetime object
  - een IncorrectDateError op te werpen indien de ingevoerde datum niet het correcte formaat heeft

Student:

- Erft over van Person.

Course:

- Beheert cursusinformatie met een naam, code, maximum en minimum aantal studenten, een lijst met Professor objecten (die de cursus geven) en een lijst met Enroll objecten (inschrijvingen voor deze cursus).
- Methods moeten professoren toevoegen en studenten inschrijven
- Overzicht methods:
  - add_professor
  - add_enrollment

Professor:

- Ook een afgeleide van Person, deze klasse bevat extra details zoals salaris en een lijst van Course objecten (cursussen die de Professor geeft).
- Het toevoegen van Courses werkt via een method

Enroll:

- Verbindt een Student met een Course, met attribuut voor het bijhouden van de inschrijvingsdatum (moment van ingave; je kan hiervoor eenvoudig datetim.now() gebruiken). Maakt gebruik van add_enrollment in Course.

<u>Aanpak voor het ontwikkelen van je onderwijsbeheersysteem</u>

Om de opdracht efficiënt aan te pakken, volg je deze stapsgewijze methode:

1. Analyse en planning
- Lees de opdracht grondig en bepaal welke klassen, methoden en attributen nodig zijn.
- Schets een overzicht van de klassen en hun relaties

2. Basisstructuur opzetten
- Maak een projectstructuur met eventueel aparte bestanden per klasse (bijvoorbeeld `person.py`, `student.py`).
- Definieer alle klassen en voeg kernattributen toe. Begin met lege methoden en gebruik `pass` waar nodig.

3. Stapsgewijs methoden implementeren
- Begin met eenvoudige methoden (zoals adressen toevoegen) en test deze direct.
- Voeg daarna complexere logica toe, zoals validatie

4. Relaties tussen klassen
- Implementeer hoe objecten samenwerken, bijvoorbeeld het koppelen van studenten aan cursussen.
- Test deze interacties, zodat je zeker weet dat alles correct werkt.

5. Geavanceerde concepten
- Voeg custom exceptions toe om fouten op te vangen, zoals een ongeldige geboortedatum.

6. Validatie en afronding
- Test je systeem grondig met normale en foutscenario's.
- Voeg duidelijke docstrings toe om je code te documenteren.

Tip: Werk iteratief en test regelmatig om problemen vroegtijdig op te lossen. Bereid een korte presentatie voor (hoeft niet schriftelijk) om je keuzes toe te lichten. Succes!